

Predicting Software Module Defects Using Static Code Metrics: A Machine Learning Analysis of the NASA KC1 Dataset

An Undergraduate Data Science Research Project (Revised Edition)

Khaled Ahmed Shehata

2026

Abstract

Software defect prediction tries to flag which parts of a codebase are likely to contain bugs before those bugs are found through testing or after release. This project examines whether static code metrics such as lines of code, cyclomatic complexity, and several Halstead measures can predict whether a software module is defective, using the NASA KC1 dataset from the PROMISE repository. A duplicate check performed on the raw 2,109-row dataset, before any cleaning, found that 897 rows were exact duplicates; these were removed before modeling, because letting the same observation appear on both sides of a train/test split can make a model look more accurate than it actually is. After cleaning, 1,212 unique modules remained, about 26% of them defective. Three classifiers, Logistic Regression, a Decision Tree, and a Random Forest, were trained and evaluated on a single stratified 75/25 split, each in a baseline version and a class-weighted version. Random Forest reached the highest ROC-AUC on this split (0.639). A Stratified 5-fold cross-validation was then run as a robustness check on the full cleaned dataset; under cross-validation, Logistic Regression's mean ROC-AUC (about 0.690) was essentially tied with, and slightly above, Random Forest's (about 0.679-0.685), which shows that the single-split ranking of models does not fully hold up once evaluation is repeated across folds. Class weighting increased recall on the defective class for Logistic Regression and the Decision Tree, but not for Random Forest, so its effect depended on the model rather than being uniform. None of the models reached strong predictive performance, and this is reported honestly rather than as a success. The main contribution of this project is a concrete, hands-on demonstration of how duplicate records and the choice of evaluation procedure can each change how trustworthy a defect-prediction result looks, more than any single prediction score.

Keywords

software defect prediction; static code metrics; NASA KC1 dataset; class imbalance; duplicate records; train/test contamination; cross-validation; machine learning; random forest; logistic regression

1. Introduction

Software testing takes time, and testing time is usually limited. If a team had even a rough idea of which modules are more likely to contain defects before testing starts, they could focus their limited testing effort where it is most needed. This idea is the basis of software defect prediction, a line of research that goes back several decades and that still appears regularly in software engineering venues.

One common approach is to use static code metrics, numbers that can be computed directly from source code without running it, such as lines of code or McCabe's cyclomatic complexity (McCabe, 1976), as input features for a machine learning model that predicts whether a module is defective. The general assumption is that larger or more complex modules are harder to get right and should therefore be more likely to contain bugs. This assumption is intuitive, but it has also been questioned in the literature (Menzies, Greenwald, & Frank, 2007), and it is one of the things this project tries to examine with real data rather than simply assume.

This paper reports a small, undergraduate-level project that applies this idea to the KC1 dataset, one of several datasets released by NASA through the PROMISE repository. Along the way, the project ran into a data quality problem that is well documented in the software engineering literature but easy to miss when working with the data for the first time: several of the NASA MDP datasets, including KC1, are known to contain a large number of duplicate records (Gray, Bowes, Davey, Sun, & Christianson, 2011; Shepperd, Song, Sun, & Mair, 2013). This project trains and evaluates its own models rather than only citing published numbers, so this duplication had to be identified and addressed directly, and doing so changed the results substantially. That experience is a central part of what this paper reports. This revised version of the paper also adds a cross-validation robustness check and corrects an earlier documentation issue in the analysis code, both described in the Methodology and Data Preprocessing sections.

2. Research Problem

Static code metrics are cheap to compute and do not require a working test suite, so if they carry a useful signal about defects, they could support practical decisions such as where to prioritize code review or testing effort. The problem is that it is not obvious in advance how strong that signal actually is, how it compares across different modeling approaches, or how much the reported strength of that signal depends on choices made during data preparation and evaluation, such as whether duplicate records are removed or whether results come from a single split or from cross-validation. This project treats these as part of the same problem: not only whether static metrics can predict defects, but also how much preprocessing and evaluation choices change the answer.

3. Research Questions

- RQ1: To what extent can static code metrics from the KC1 dataset predict whether a software module is defective?
- RQ2: How do Logistic Regression, a Decision Tree, and a Random Forest compare on this task, using accuracy, precision, recall, F1-score, and ROC-AUC?
- RQ3: Does applying class weighting to address the imbalance between defective and non-defective modules change model behavior, and if so, in what way?
- RQ4: How does removing duplicate records from KC1 affect the reported performance of these models compared to using the raw dataset?

4. Objectives of the Study

- Load, inspect, and clean the KC1 dataset, and document all data quality issues found in the process.
- Carry out an exploratory data analysis of the cleaned dataset.
- Train and compare three classification models under two class-imbalance handling strategies.
- Evaluate all models using metrics appropriate for an imbalanced classification problem, not accuracy alone.
- Check how much the single train/test split affects the results, using Stratified 5-fold cross-validation.
- Identify which static metrics contribute most to the Random Forest's predictions, without treating this as evidence of causation.
- Compare results before and after duplicate removal, to illustrate the practical impact of this specific data quality problem.

5. Related Work

Static code metrics used in defect prediction mostly trace back to two sources: McCabe's cyclomatic complexity, which counts the number of independent paths through a program's control flow graph (McCabe, 1976), and Halstead's software science metrics, derived from counts of operators and operands in the source code (Halstead, 1977). Both were originally proposed as general indicators of program complexity, not specifically as defect predictors, but were later adopted for that purpose because they can be computed automatically.

Menzies, Greenwald, and Frank (2007) is one of the more influential studies using this type of data. They trained defect predictors on several NASA datasets, including KC1, and reported that fairly simple learners could reach reasonable detection rates, while cautioning that the relationship between static metrics and defects is not fully understood and should not be overstated. Lessmann, Baesens, Mues, and Pietsch (2008) later ran a large benchmark comparing many classifiers, including logistic regression and ensemble methods, across several NASA datasets, and found that classifier choice often mattered less than expected once performance was measured with a threshold-independent metric such as ROC-AUC. Hall, Beecham, Bowes, Gray, and Counsell (2012) reviewed a large number of fault prediction studies and reported that reported performance varies considerably across studies, partly because of differences in how datasets are prepared.

The data quality of the NASA MDP datasets specifically has been questioned more than once. Gray, Bowes, Davey, Sun, and Christianson (2011) documented multiple problems in these datasets, including duplicate and inconsistent records, and argued that the datasets require substantial cleaning before use in defect prediction experiments. Shepperd, Song, Sun, and Mair (2013) compared different versions of the NASA datasets in circulation, again identifying data quality issues and recommending that researchers report preprocessing steps in enough detail to be reproduced. This project's own finding, that 897 of 2,109 KC1 records were exact duplicates, is consistent with what these two papers describe. The decision to remove duplicates before modeling follows their general recommendation, applied at a scale appropriate for an undergraduate project rather than a full replication of their more detailed cleaning procedures.

On the modeling side, Random Forest (Breiman, 2001) is used here as the ensemble method, since it is one of the most commonly used classifiers in this line of research and is reasonably robust to noisy features without extensive tuning. All models were implemented using scikit-learn (Pedregosa et al., 2011). SMOTE (Chawla, Bowyer, Hall, & Kegelmeyer, 2002) is referenced in the Future Work section as a possible alternative to class weighting for handling imbalance, but was not used in the experiments reported here.

6. Dataset Description

The dataset used in this project is KC1, part of the NASA Metrics Data Program (MDP) collection, donated to the PROMISE Software Engineering Repository in December 2004 by Tim Menzies. It was obtained as an ARFF file from a public GitHub mirror of the PROMISE repository (ApoorvaKrisna/NASA-promise-dataset-repository), and its structure and row count were cross-checked against the same dataset hosted on OpenML (dataset ID 1067) to confirm consistency before use. Both sources are listed in the References section.

The raw dataset contains 2,109 software modules described by 21 numeric static code metrics, listed in Table 1, plus a binary label indicating whether the module has one or more known defects. All 21 features come from the McCabe and Halstead families of metrics described in the Related Work section.

Table 1. Features in the KC1 dataset

Feature	Description
loc	Lines of code (McCabe count)
v(g)	Cyclomatic complexity
ev(g)	Essential complexity
iv(g)	Design complexity
n	Halstead total operators + operands
v	Halstead volume
l	Halstead program length
d	Halstead difficulty
i	Halstead intelligence content
e	Halstead effort
b	Halstead estimated number of delivered bugs
t	Halstead time estimator
lOCode	Lines of code (Halstead count)
lOComment	Lines of comments
lOBlank	Blank lines
locCodeAndComment	Lines with both code and comment
uniq_Op	Number of unique operators
uniq_Opnd	Number of unique operands
total_Op	Total operators
total_Opnd	Total operands
branchCount	Number of branches in the control flow graph

The target variable is defects, a binary label (true/false) indicating whether the module was found to contain one or more defects.

7. Data Preprocessing

The dataset was loaded from the ARFF file into a pandas DataFrame, and the byte-string class labels were decoded into a clean text column, later converted to a binary integer label (1 for defective, 0 for non-defective). No missing values were found in any of the 21 features or the label column, and all features are numeric, so no categorical encoding was needed.

A duplicate check, run on the raw data before any rows were removed, found that 897 of the 2,109 rows (about 43%) were exact duplicates of another row already present in the dataset (identical across all 21 features and the label). This is consistent with what Gray et al. (2011) and Shepperd et al. (2013) report about the NASA MDP datasets more generally. An earlier version of the analysis script computed this duplicate count after already removing duplicates,

which meant the script's own printed output reported zero duplicates on the cleaned data, even though the removal step had genuinely removed 897 rows beforehand. This was an order-of-operations mistake in the code, not an error in the removal itself, but it meant the script did not independently reproduce the 897 figure reported in the text. The corrected script used for this revision computes the duplicate count on the raw data first, and only removes duplicates afterward; re-running it reproduces the same figure (897) directly from the data rather than relying on a number written into the report.

Keeping duplicate rows would mean that, after a random train/test split, some test modules could be exact copies of a module already seen during training. A model could then appear to perform well on the test set partly by having already seen an identical row during training, rather than by learning a real relationship between the code metrics and the defect label. This project refers to this as duplicate-induced train/test contamination. It behaves similarly to data leakage in its effect, since a model can be evaluated on an observation identical to one it already saw in training, which can produce overly optimistic estimates of generalization, but it differs from the more common definition of leakage, which usually involves information about the target variable being encoded into a feature. Because of this distinction, this paper uses the more specific term throughout rather than 'data leakage' alone.

To address this, all 897 fully duplicated rows were removed, keeping the first occurrence of each unique row. This reduced the dataset from 2,109 to 1,212 unique modules. The class balance also changed: the raw dataset was 84.5% non-defective and 15.5% defective, while the deduplicated dataset is 74.0% non-defective and 26.0% defective (897 non-defective, 315 defective). This happened because a large share of the duplicated rows were small, simple modules that were mostly non-defective, so removing them reduced the non-defective majority more than the defective minority in relative terms. This deduplicated dataset of 1,212 modules is the primary dataset used for every result reported in this paper.

As a methodological check, the same modeling pipeline was also run once on the raw, non-deduplicated 2,109-row dataset, to observe the effect of the duplicates on reported performance. That run produced noticeably higher metrics across all models. This comparison, reported in the Results section, is included only as a methodological observation about the effect of duplicate records in this particular dataset and experiment. It should not be read as a general claim that duplicate removal changes results by a similar margin on every dataset, or that a model trained on the raw data is simply 'wrong'; rather, the point is that a random split becomes less trustworthy as an evaluation procedure when identical observations can end up on both sides of it.

All 21 features were standardized (zero mean, unit variance) using scikit-learn's StandardScaler before being passed to any model. This was implemented as part of an explicit scikit-learn Pipeline for each model, so the scaler is always fit only on training data, whether that is the training portion of the single 75/25 split or the training fold during cross-validation; the held-out portion is only ever transformed, never used to fit the scaler. Standardization matters most for Logistic Regression, which is sensitive to the scale of input features. It is not strictly necessary for the Decision Tree or Random Forest models, since tree-based splits are based on feature thresholds and are not affected by a monotonic rescaling of a feature. Scaling was nonetheless applied uniformly to all three models, mainly for implementation simplicity and to keep preprocessing identical across models; because rescaling does not change the relative order of values used for tree splits, it is not expected to have materially changed the Decision Tree or Random Forest results, though it was not re-verified by running the tree models on unscaled data.

8. Methodology

The analysis was implemented in Python using pandas and numpy for data handling, scikit-learn for modeling and evaluation, and matplotlib for visualization (Pedregosa et al., 2011). After cleaning, the 21 static code metrics were used as input features (X) and the binary defects label as the target (y).

The primary evaluation used a single stratified train/test split: 75% training (909 modules) and 25% test (303 modules), stratified so that the proportion of defective modules stayed approximately the same in both sets (about 26%). A fixed random seed (42) was used for this split and throughout the rest of the analysis, for reproducibility.

Three classifiers were trained: Logistic Regression, a Decision Tree, and a Random Forest with 200 trees. Each was trained twice, once with scikit-learn's default settings (the baseline version) and once with `class_weight='balanced'`, which reweights the loss function so that errors on the minority (defective) class count for more during training. No hyperparameter tuning was performed beyond this; all other settings were left at scikit-learn's defaults. Comparing baseline and balanced versions of the same model makes it possible to see the precision/recall trade-off introduced by class weighting directly, without changing anything else about the model.

To check how much the results depended on this specific 75/25 split, a Stratified 5-fold cross-validation was also run on the full cleaned dataset (1,212 modules) for all six model configurations. Cross-validation divides the data into five folds of roughly equal size, each preserving the overall 74/26 class balance, trains each model on four folds, and evaluates it on the remaining fold, repeating this five times so every fold is used once for evaluation. Folds were shuffled before splitting, using the same fixed random seed (42). For each metric (accuracy, precision, recall, F1, ROC-AUC), the mean and standard deviation across the five folds are reported. Because the StandardScaler step is inside the same Pipeline used for cross-validation, it is refit separately within each fold on that fold's training portion only, so no information from a fold's held-out portion is used to fit the scaler.

9. Exploratory Data Analysis

After cleaning, the dataset is imbalanced, with 897 non-defective modules (74.0%) and 315 defective modules (26.0%), shown in Figure 1.

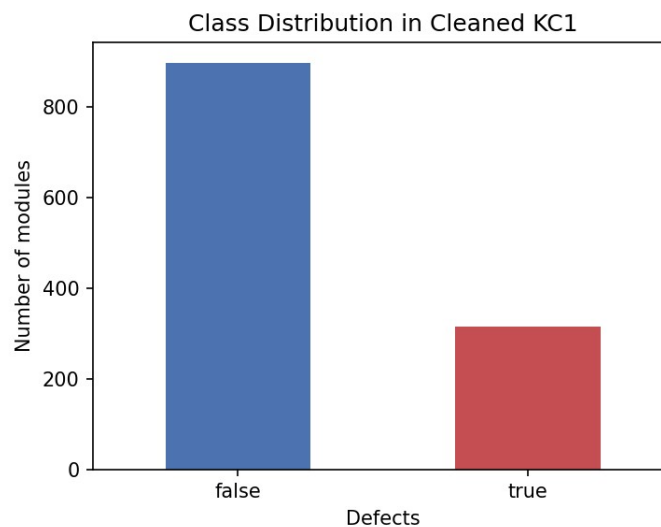


Figure 1. Class distribution in the cleaned KC1 dataset ($n = 1,212$).

Most of the static metrics are heavily right-skewed: the majority of modules are small and simple, with a smaller number of much larger, more complex modules. For example, the median value of `loc` is 19 lines, but the maximum is 288 lines, and the mean (31.5) sits well above the median, which is typical of this kind of skewed distribution. Figure 2 compares several key metrics between defective and non-defective modules using boxplots. In most of these metrics, defective modules tend to have a higher median and a wider spread than non-defective ones, consistent with the general idea that larger, more complex modules are somewhat more likely to be defective, though the two distributions overlap substantially.

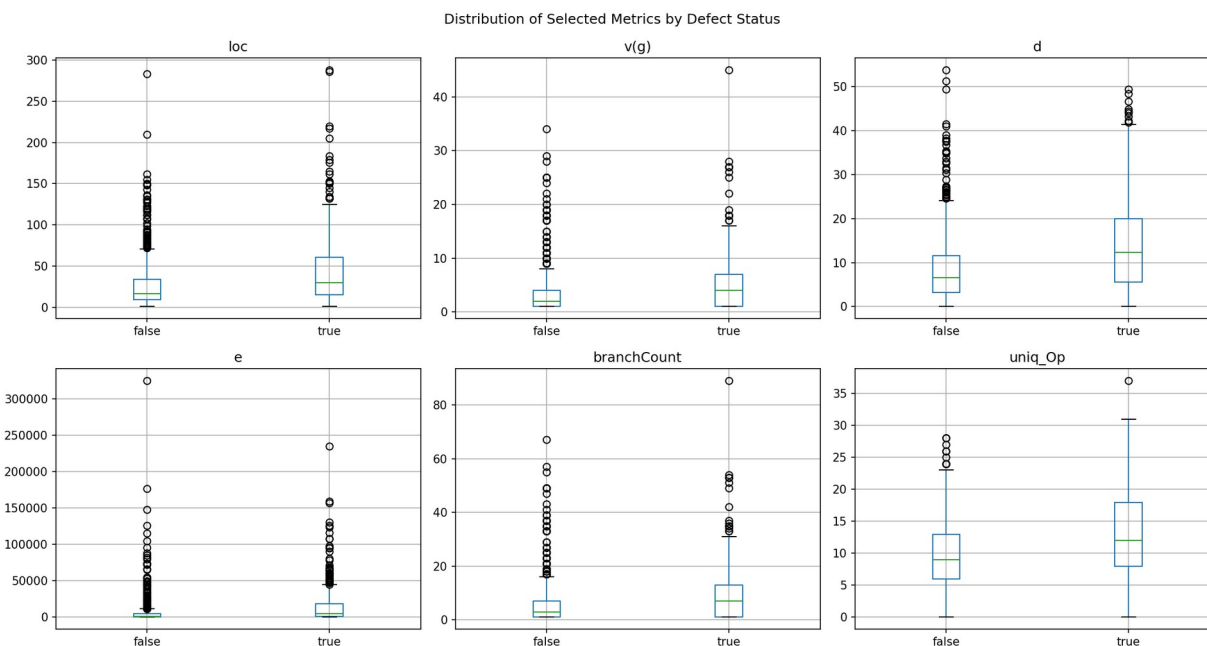


Figure 2. Distribution of selected static code metrics by defect status.

Table 2 reports the Pearson correlation between each feature and the binary defect label. The strongest associations, all modest, come from Halstead difficulty (`d`) and the counts of unique operands and operators (`uniq_Opnd`, `uniq_Op`). Two features, `locCodeAndComment` and `l` (Halstead program length), show essentially no linear relationship or a weak negative one with the label. None of these correlations is strong enough on its own to be a reliable defect indicator, which motivates using a model that can combine several features rather than relying on any single metric.

Table 2. Correlation of each feature with the defect label (deduplicated data)

Feature	Correlation with defect label
<code>d</code> (Halstead difficulty)	0.280
<code>uniq_Opnd</code>	0.278
<code>uniq_Op</code>	0.273
<code>total_Opnd</code>	0.266
<code>n</code>	0.257
<code>total_Op</code>	0.250
<code>v</code> (Halstead volume)	0.250
<code>b</code>	0.249
<code>lOBlank</code>	0.245
<code>loc</code>	0.244
<code>v(g)</code> (cyclomatic complexity)	0.204

Feature	Correlation with defect label
locCodeAndComment	-0.037
l (Halstead program length)	-0.136

Figure 3 shows the full correlation matrix among all 21 features. Several McCabe and Halstead metrics are strongly correlated with each other, which is expected, since many are different mathematical transformations of the same underlying counts of code size and operators/operands.

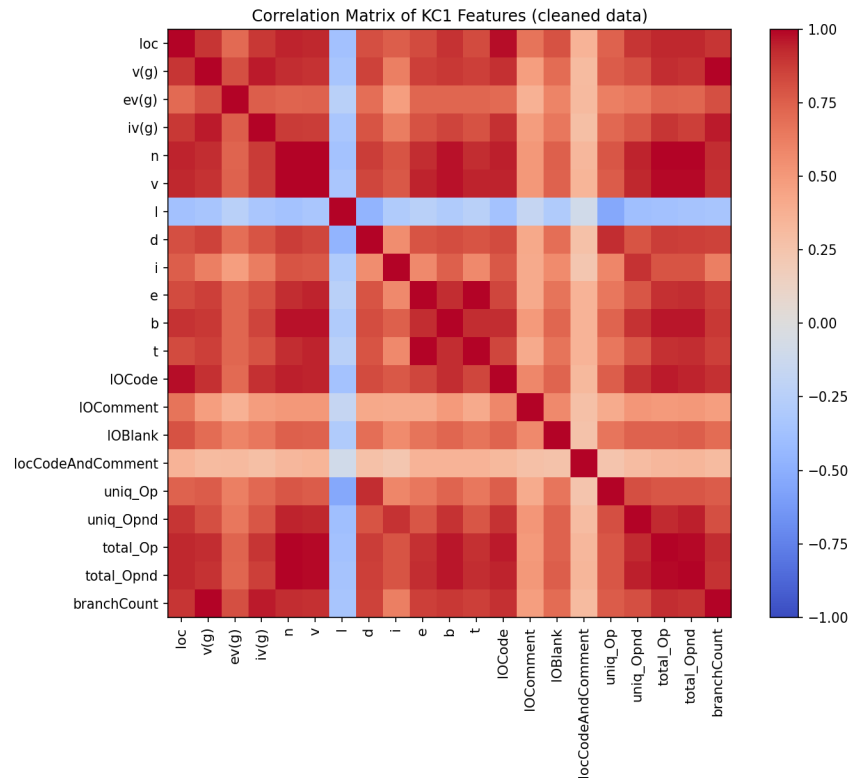


Figure 3. Correlation matrix of the 21 static code metrics.

10. Machine Learning Models

Three classification models were used, representing a linear model, a single interpretable tree, and an ensemble of trees:

- **Logistic Regression:** a linear model that estimates the probability of a module being defective from a weighted combination of the input features. It is simple, fast, and interpretable, which makes it a reasonable baseline.
- **Decision Tree:** a single tree that splits the data on feature thresholds. It can capture non-linear relationships and feature interactions, but a single unrestricted tree tends to overfit, which is worth keeping in mind when reading its results.
- **Random Forest:** an ensemble of 200 decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split (Breiman, 2001). Averaging over many trees usually reduces the overfitting problem of a single decision tree.

Each model was trained in a baseline version and a class-weighted version using `class_weight='balanced'`, a standard and relatively simple way of handling class imbalance appropriate for an undergraduate project, in contrast to more

advanced resampling techniques such as SMOTE (Chawla et al., 2002), which is discussed as a possible extension in the Future Work section but was not used here.

11. Experimental Setup

- Dataset: cleaned KC1, 1,212 modules, 21 features, binary label.
- Primary split: stratified 75/25 train/test split (909 training, 303 test modules), random_state = 42.
- Robustness check: Stratified 5-fold cross-validation on the full cleaned dataset (1,212 modules), shuffle = True, random_state = 42.
- Preprocessing: StandardScaler, fit only inside each training split/fold via an sklearn Pipeline, never on held-out data.
- Models: Logistic Regression, Decision Tree, Random Forest (200 trees), each in a baseline and a class-weighted version, six configurations in total.
- Hyperparameters: scikit-learn defaults throughout; no tuning performed, aside from setting Random Forest to 200 trees and toggling class_weight.
- Evaluation metrics: Accuracy, Precision, Recall, F1-score, and ROC-AUC, computed with respect to the defective (positive) class.

Accuracy alone is not a reliable metric here because the dataset is imbalanced: a model that always predicted "not defective" would already reach roughly 74% accuracy on the test set without identifying a single defective module. Precision and recall focus specifically on the defective class: precision measures how many of the modules flagged as defective actually are defective, while recall measures how many of the truly defective modules the model successfully identifies. F1-score summarizes precision and recall as a single number, and ROC-AUC measures how well the model separates the two classes across all possible decision thresholds, independent of the specific threshold used to turn a predicted probability into a yes/no prediction.

12. Data Analysis and Results

12.1 Single Train/Test Split

Table 3 reports the results of all six models on the held-out test set (303 modules, 79 defective).

Table 3. Model performance on the single test split (deduplicated KC1, n = 303)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (baseline)	0.739	0.500	0.228	0.313	0.634
Logistic Regression (balanced)	0.634	0.360	0.519	0.425	0.637
Decision Tree (baseline)	0.634	0.318	0.354	0.335	0.540
Decision Tree (balanced)	0.663	0.379	0.456	0.414	0.598
Random Forest (baseline)	0.720	0.446	0.317	0.370	0.639
Random Forest (balanced)	0.713	0.423	0.279	0.336	0.637

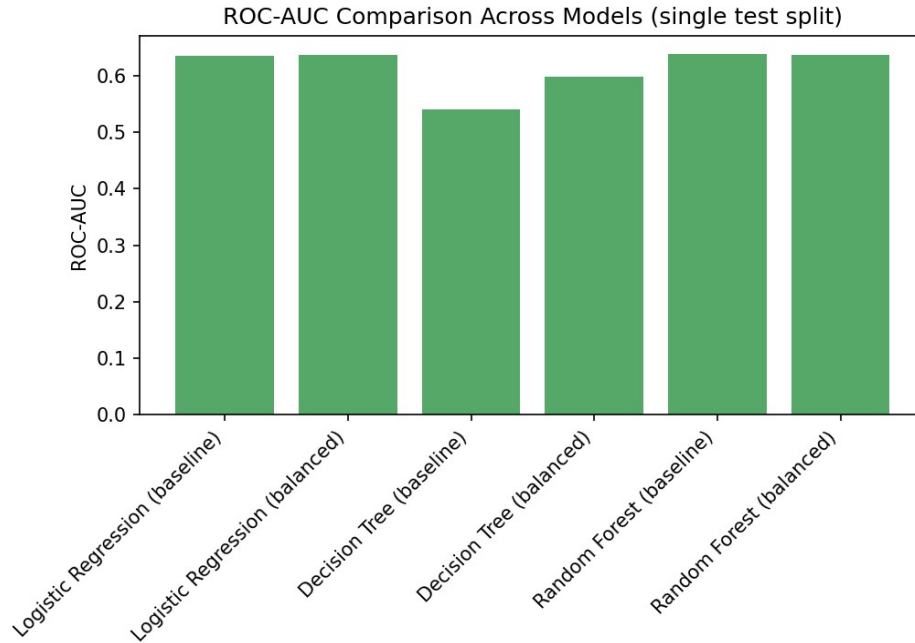


Figure 4. ROC-AUC comparison across all six models (single test split).

The baseline Logistic Regression illustrates clearly what an imbalanced dataset does to a model that is not adjusted for it: accuracy looks reasonably high (73.9%), but recall on the defective class is only 22.8%, meaning the model misses more than three out of four actually defective modules. Applying class weighting raised recall to 51.9%, at the cost of accuracy dropping to 63.4% and precision dropping to 36.0%, and F1 improving from 0.313 to 0.425. This is the expected precision/recall trade-off: the balanced model flags more modules as defective overall, which catches more true defects but also produces more false alarms.

The Decision Tree performed the weakest of the three models by ROC-AUC on this split (0.540 baseline, 0.598 balanced), consistent with a single tree being prone to overfitting the training data. The balanced Decision Tree showed a clear improvement over its baseline on both recall (0.456 vs. 0.354) and F1-score (0.414 vs. 0.335).

Random Forest achieved the highest ROC-AUC of the three models on this split, in both its baseline (0.639) and balanced (0.637) versions, though the difference between these two is small. Unlike Logistic Regression, class weighting did not clearly improve Random Forest's recall on this split; recall was actually slightly lower in the balanced version (0.279 vs. 0.317 for the baseline). Taken together, these three models show that class weighting had a model-dependent effect in this experiment rather than a uniformly beneficial one: it produced a clear, expected precision/recall trade-off for Logistic Regression and the Decision Tree, but did not provide a clear benefit for Random Forest.

12.2 Cross-Validation Robustness Check

Table 5 reports the Stratified 5-fold cross-validation results on the full cleaned dataset (1,212 modules), with the mean and standard deviation of each metric across the five folds.

Table 5. Stratified 5-fold cross-validation results (cleaned KC1, $n = 1,212$; mean $\pm$ SD across folds)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (baseline)	0.760 ± 0.009	0.620 ± 0.057	0.203 ± 0.029	0.305 ± 0.034	0.690 ± 0.018
Logistic Regression (balanced)	0.682 ± 0.031	0.420 ± 0.032	0.565 ± 0.072	0.479 ± 0.037	0.690 ± 0.017
Decision Tree (baseline)	0.666 ± 0.023	0.365 ± 0.041	0.384 ± 0.061	0.373 ± 0.045	0.573 ± 0.033
Decision Tree (balanced)	0.654 ± 0.023	0.341 ± 0.027	0.343 ± 0.024	0.340 ± 0.011	0.555 ± 0.010
Random Forest (baseline)	0.749 ± 0.012	0.539 ± 0.052	0.295 ± 0.033	0.379 ± 0.025	0.685 ± 0.033
Random Forest (balanced)	0.751 ± 0.021	0.553 ± 0.091	0.292 ± 0.037	0.378 ± 0.035	0.679 ± 0.035

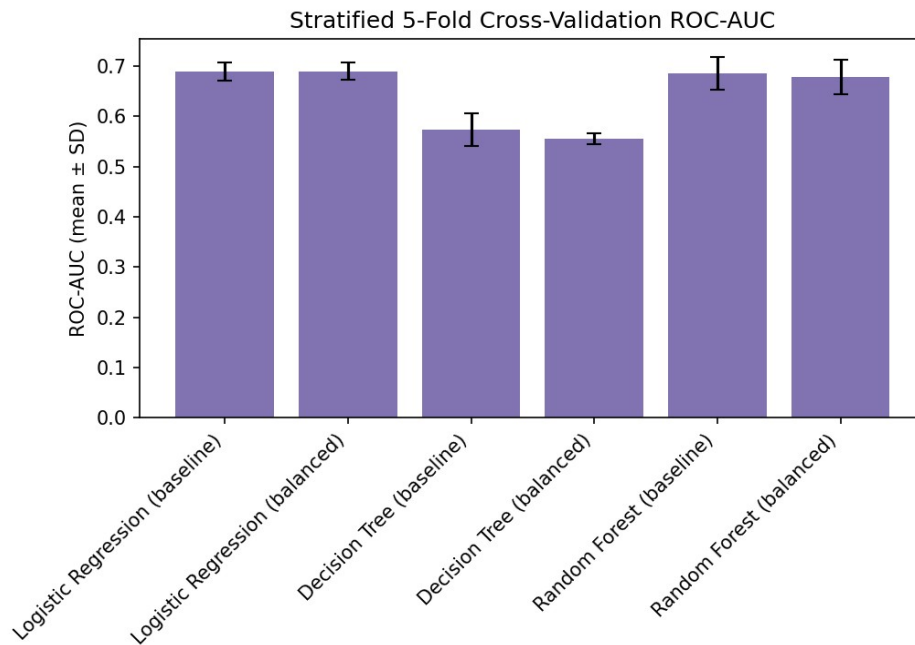


Figure 5. Cross-validation ROC-AUC (mean ± SD) across models.

The cross-validation results are broadly consistent with the single split in overall shape, but they change the ranking between the two strongest models. On the single split, Random Forest (baseline) had the highest ROC-AUC (0.639) compared to Logistic Regression (baseline, 0.634). Under 5-fold cross-validation, this ordering reverses slightly: Logistic Regression's mean ROC-AUC (0.690 for both baseline and balanced) is marginally higher than Random Forest's (0.685 baseline, 0.679 balanced). The two are close enough, especially given the standard deviations (around 0.02-0.03 for both), that this project does not treat either model as clearly superior; the more accurate statement is that Logistic Regression and Random Forest perform similarly on this task once evaluation is repeated across multiple folds, and that the specific 75/25 split used for the primary analysis happened to favor Random Forest slightly. This is itself a useful finding: it shows that a conclusion based on a single split ('Random Forest is best') is not fully robust, and that repeating the evaluation changes the picture in a way that a single split cannot reveal on its own.

The Decision Tree remained the weakest model under cross-validation as well (mean ROC-AUC 0.573 baseline, 0.555 balanced), consistent with the single-split result. Class weighting's effect on recall was also broadly consistent with the

single split: it clearly increased recall for Logistic Regression (0.203 to 0.565) and left it roughly unchanged for the Decision Tree and Random Forest, matching the model-dependent pattern already described in section 12.1.

12.3 Feature Importance

Figure 6 shows the ten most important features according to the balanced Random Forest model fit on the single train/test split, using scikit-learn's built-in feature importance, which measures how much each feature contributed, on average, to reducing impurity across the trees in the forest. The most important features were *i* (Halstead intelligence content), *v* (Halstead volume), *loc*, *d* (Halstead difficulty), and *e* (Halstead effort). This measure reflects the contribution of a feature to this specific Random Forest model's decision process under this experimental setup, and should not be interpreted as evidence that these metrics cause defects; several of these features are also correlated with each other, as shown in Figure 3, which can spread importance across them and make individual rankings less stable than they might appear.

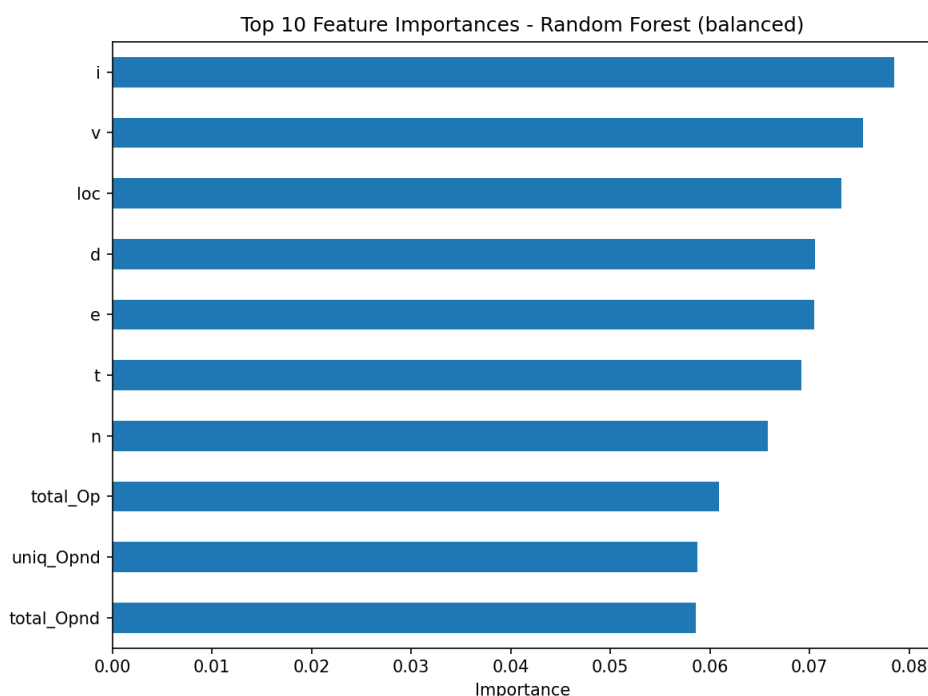


Figure 6. Top 10 feature importances from the Random Forest (balanced) model, single split.

12.4 Effect of Duplicate Removal (Methodological Comparison)

As described in the Data Preprocessing section, the same single-split modeling pipeline was also run once on the raw, non-deduplicated 2,109-row dataset, to observe the effect of the 897 duplicate rows. Table 4 compares the Random Forest (baseline) results on both versions of the data.

Table 4. Random Forest (baseline) results with and without duplicate removal

Dataset version	n	Accurac y	Precisio n	Recall	F1	ROC- AUC
Raw KC1 (with duplicates)	2,109	0.864	0.600	0.366	0.455	0.836
Cleaned KC1 (deduplicated)	1,212	0.720	0.446	0.317	0.370	0.639

Every metric is higher on the raw, non-deduplicated data, and the gap in ROC-AUC (0.836 versus 0.639) is particularly large. This is consistent with the duplicate-induced contamination described earlier: when duplicate rows are split across the training and test sets, some test modules are identical to modules the model has already seen, which inflates every performance metric without the model having learned anything more generalizable. This does not mean the raw-data model is simply wrong; rather, it means that a random split on data containing many duplicates is a less trustworthy evaluation procedure, because it does not reliably measure how the model would perform on genuinely new, unseen modules. This comparison is included only to illustrate the size of this effect in this specific dataset and experiment, and should not be read as a general claim that data cleaning always changes results by a similar margin on other datasets.

13. Discussion

Returning to RQ1, static code metrics on their own show a real but limited ability to predict whether a KC1 module is defective. The best mean ROC-AUC obtained on the cleaned dataset under cross-validation (about 0.69, Logistic Regression) is clearly above the 0.5 expected from random guessing, so the metrics carry some signal, but it is a modest one. This is broadly consistent with Menzies et al. (2007) and Lessmann et al. (2008): static metrics are useful but imperfect predictors, and defect prediction in general does not currently reach a level of accuracy that would let it fully replace manual code review or testing decisions.

On RQ2, the three models performed differently but not dramatically so, and the ranking between the two strongest models was not stable across evaluation procedures. Random Forest had the highest ROC-AUC on the single train/test split, but Logistic Regression had a very similar, and in the cross-validation results marginally higher, mean ROC-AUC once the evaluation was repeated across five folds. The more defensible conclusion is that Random Forest and Logistic Regression perform comparably on this dataset, and that a simpler, more interpretable linear model does about as well as a more complex ensemble here. The Decision Tree was consistently the weakest model under both evaluation procedures, matching its known tendency to overfit when grown without restrictions.

On RQ3, class weighting had a model-dependent effect rather than a uniformly beneficial one, and this held under both the single split and cross-validation. For Logistic Regression, weighting produced a clear trade-off: recall increased substantially (from about 0.20-0.23 to 0.52-0.57 depending on evaluation procedure), while precision and accuracy decreased and F1 improved. For the Decision Tree, weighting also increased recall and F1 on the single split, though this was less pronounced under cross-validation. For Random Forest, weighting did not provide a clear benefit under either evaluation procedure: ROC-AUC stayed almost unchanged, and recall was slightly lower in the balanced version than in the baseline. The results suggest that the effect of class weighting depended on the classifier and was not uniformly beneficial under this experimental setup. One possible explanation is that Random Forest's own averaging over many trees already smooths out some of the imbalance's effect, so an explicit class weight adjustment has less additional impact than it does for a linear model; this is offered as a plausible hypothesis based on the pattern observed here, not as an established finding, since it was not directly tested.

RQ4 turned out to be the most consequential part of this project. Simply keeping the 897 duplicate rows in the dataset would have led to a considerably more optimistic story: ROC-AUC around 0.84 for Random Forest instead of 0.64, and similarly inflated numbers for the other metrics. This is a direct, hands-on illustration of a point that is easy to read about in a methodology textbook but easy to underestimate until it shows up in one's own results, and it matches what Gray et al. (2011) and Shepperd et al. (2013) found when auditing these datasets more thoroughly than this project attempted to.

A second, related methodological point came from the cross-validation check: it showed that even the cleaned dataset's single-split ranking of models (Random Forest as the clear best) does not fully hold up once the evaluation is repeated across folds, where Logistic Regression performs about as well or marginally better. This is a smaller effect than the duplicate-removal difference, but it reinforces the same underlying lesson, that a single evaluation number, whether from one train/test split or from one uncleaned dataset, can be more or less trustworthy depending on how it was produced, and that checking this is itself a meaningful part of the analysis.

14. Limitations

- Dataset size: after removing duplicates, only 1,212 modules remain, and the single test split contains only 79 defective modules. This is a small sample for estimating minority-class metrics like recall and precision reliably. Cross-validation reduces, but does not eliminate, this limitation, since it is still based on the same 1,212 modules; it shows how much results vary across different splits of this same data, not how the models would perform on a larger or different dataset.
- Historical and single-project data: KC1 comes from one NASA software project using development practices and a programming environment from the 1990s/2000s. Results may not generalize to modern software projects, different programming languages, or different development processes.
- Limited feature set: only static McCabe and Halstead metrics were available. Information that might be genuinely predictive of defects, such as change history, developer experience, test coverage, or code review activity, was not part of this dataset and could not be included.
- Correlation, not causation: the correlations reported in the EDA and the Random Forest feature importances describe association, not a causal relationship. A module having a high value for a given metric does not mean that metric caused it to be defective.
- Residual data quality concerns: this project only removed exact full-row duplicates. Gray et al. (2011) and Shepperd et al. (2013) describe additional, more subtle quality issues in the NASA MDP datasets (such as inconsistent or implausible values) that were not investigated here and could still be affecting the results.
- No hyperparameter tuning: all three models were run with scikit-learn's default settings (aside from the number of trees in the Random Forest and the `class_weight` toggle). Reported performance reflects untuned models and may not represent the best each model could achieve.
- Uniform scaling across model types: all models were trained on standardized features for implementation consistency, even though standardization is not required for the tree-based models. This is not expected to have materially affected the Decision Tree or Random Forest results, but this was not directly verified by comparing scaled and unscaled versions of these two models.

15. Conclusion

This project used the NASA KC1 dataset to test whether static code metrics can predict software defects, comparing three machine learning models under two class-imbalance strategies and two evaluation procedures. After identifying and removing 897 duplicate records, an evaluation on 1,212 unique modules showed modest but real predictive signal in the static metrics. Random Forest had the best ROC-AUC on the single train/test split (0.639), but a 5-fold cross-validation check showed that Logistic Regression performed comparably, and marginally better on average (about 0.690), so the difference between the two strongest models is not large or fully stable across evaluation procedures.

Class weighting had different effects depending on the model: it clearly increased recall on the defective class for Logistic Regression and, to a lesser extent, the Decision Tree, but did not provide a clear benefit for Random Forest. Duplicate records had a substantial impact on evaluation results, with the raw, non-deduplicated dataset producing a ROC-AUC roughly 0.2 higher than the cleaned dataset for the same model. Taken together, these findings support a

conclusion that is realistic rather than impressive: static code metrics contain some predictive signal about defects, model choice matters less than expected once evaluation is done carefully, and how the data is prepared and evaluated can affect the results as much as, or more than, which model is used. The strongest contribution of this project is this practical demonstration of how data quality and evaluation design affect defect-prediction results, rather than any single prediction score.

16. Future Work

- Compare this simple class-weighting approach against resampling techniques such as SMOTE (Chawla et al., 2002), which generates synthetic minority-class examples rather than reweighting the existing ones.
- Apply the more thorough data cleaning rules proposed by Shepperd et al. (2013) and Gray et al. (2011), which go beyond exact duplicate removal to address other data quality issues in the NASA MDP datasets.
- Perform basic hyperparameter tuning (for example, a grid or random search over Random Forest depth and number of trees, or Logistic Regression's regularization strength) to see how much of the gap to a truly optimized model remains.
- Repeat this analysis on one or more of the other PROMISE/NASA datasets (such as PC1 or CM1) to see whether the same patterns, including the duplicate-record issue and the closeness between Logistic Regression and Random Forest, appear elsewhere.
- If access to a more modern, actively maintained codebase becomes available, repeat a similar analysis using more contemporary metrics (for example, churn-based or process metrics) alongside or instead of the McCabe/Halstead metrics used here.

17. References

- ApoorvaKrisna. NASA-promise-dataset-repository [Data set]. GitHub. Retrieved from <https://github.com/ApoorvaKrisna/NASA-promise-dataset-repository/blob/main/kc1.arff>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Gray, D., Bowes, D., Davey, N., Sun, Y., & Christianson, B. (2011). The misuse of the NASA metrics data program data sets for automated software defect prediction. In *Proceedings of the 15th Annual Conference on Evaluation & Assessment in Software Engineering (EASE 2011)* (pp. 96–103). IET.
- Hall, T., Beecham, S., Bowes, D., Gray, D., & Counsell, S. (2012). A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering*, 38(6), 1276–1304.
- Halstead, M. H. (1977). *Elements of software science*. New York: Elsevier.
- Lessmann, S., Baesens, B., Mues, C., & Pietsch, S. (2008). Benchmarking classification models for software defect prediction: A proposed framework and novel findings. *IEEE Transactions on Software Engineering*, 34(4), 485–496.

- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
- Menzies, T., Greenwald, J., & Frank, A. (2007). Data mining static code attributes to learn defect predictors. *IEEE Transactions on Software Engineering*, 33(1), 2–13.
- OpenML. KC1 [Data set, ID 1067]. Retrieved from <https://www.openml.org/search?id=1067&type=data>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Shepperd, M., Song, Q., Sun, Z., & Mair, C. (2013). Data quality: Some comments on the NASA software defect datasets. *IEEE Transactions on Software Engineering*, 39(9), 1208–1215.

Appendix A. Revision Notes

This appendix documents the changes made in this revised edition of the paper, for transparency and reproducibility.

A.1 Code correction

The analysis script originally removed duplicate rows before computing the duplicate count, so its own printed output reported zero duplicates on the already-cleaned data. The corrected script computes the duplicate count on the raw, unmodified data first (897 duplicates, verified again in this revision) and only removes duplicates afterward. The single train/test split results were re-generated with the corrected script and are numerically identical to the results reported in the earlier version of this paper, confirming that the underlying modeling results were not affected by this documentation issue.

A.2 New analysis added

A Stratified 5-fold cross-validation (`shuffle = True`, `random_state = 42`) was added as a genuine robustness check on the full cleaned dataset, using scikit-learn Pipelines so that feature scaling is fit separately within each fold. This was executed, not estimated or fabricated; the results are reported in Table 5 and Figure 5, and are discussed in sections 12.2, 13, and 15.

A.3 Wording and terminology corrections

- Replaced the unqualified claim that class weighting "consistently increased recall" with a model-specific description (clear increase for Logistic Regression and, to a lesser extent, the Decision Tree; no clear benefit for Random Forest).
- Replaced the term "data leakage" on its own with "duplicate-induced train/test contamination," while explaining why it behaves similarly to leakage in effect.
- Clarified that removing duplicates made the raw model's evaluation less trustworthy, rather than describing the raw model itself as incorrect.
- Added an explicit statement that any explanation offered for why class weighting affected Random Forest differently is a hypothesis, not an established finding.
- Added a dataset citation for the GitHub/PROMISE source and the OpenML mirror used to verify it.

- Updated the Limitations section to reflect the addition of cross-validation, and added limitations regarding the absence of hyperparameter tuning and the uniform application of scaling across all model types.